Freight Rate Prediction

Spotter Machine Learning Engineer assessment | 31 October 2026

1. Summary

- Task: predict posted_rate for 12,000 loads in Nov-Dec 2025, learning from 48,000 labelled loads in Jan-Oct 2025.
- Validation: three rolling, time-based folds that each train on the past and predict the next two months, the same shape as the real task. The selected model averages $44 MAE (1.9% MAPE) on clean test loads, with a worst fold of $51; a naive $/mile baseline scores $106.
- Model: an average of LightGBM (70%) and a log-linear model (30%) on log(rate), plus a market price level shared by all loads on a day (recent level + trend + quarter-end ramp). The market level is the biggest gain: LightGBM alone scores $77.
- Key data findings: quote_signal leaks the answer in some months, mirrors it in others and is pure noise in Nov-Dec, so it is not used; 677 broken prices (1.4%) are excluded from training; eight validation cities never appear in training and are priced from coordinates.

2. Data and data quality

train_test.csv holds 48,000 loads from 1 January to 31 October 2025 with the posted rate. validation.csv holds 12,000 loads from 1 November to 31 December 2025 without it. Rates are close to multiplicative in their drivers (shorter trips cost more per mile; Reefer > Flatbed > Dry Van; heavier loads cost slightly more), so all models work on log(rate).

Issue	train_test	validation	How it was handled
Negative weight (sign error)	292	145	Absolute value
Missing weight	300	165	Median for the equipment type + a missing flag
Missing market_index	374	249	Median market_index of the same day
Broken prices (1.8x+ off expected)	677	-	Excluded from training; reported in raw test scores
quote_signal unreliable / leaky	all rows	all rows	Not used as a feature (section 3.4)
Cities not seen in training (8)	-	1447	Model uses coordinates, not city names
December file lacks coordinates and market_index	-	31 rows	City coordinate lookup; daily market_index from validation inputs

Broken prices were found by comparing each load's log rate with a robust log-linear estimate. Normal loads sit within about ±15% of it, the broken ones 2-7x away, with no loads in between, so the 1.8x cut-off is not sensitive.

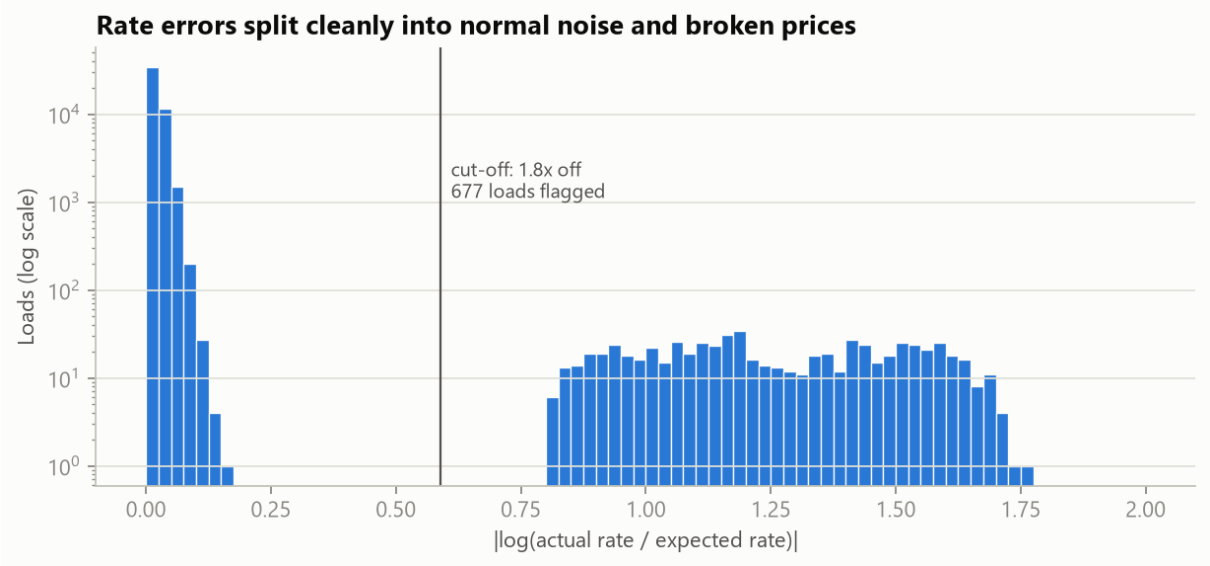


Figure 1. Distance of each load's rate from its expected rate (log scale).

3. Validation approach and data split

3.1 Why the split is by time, not random

The real task is a forecast: train on Jan-Oct and predict Nov-Dec. A random split puts loads from the same days in both train and test, so the model is tested on market conditions it has already seen, and it rewards features like quote_signal whose meaning changes over time. The table compares the same LightGBM model (without the market level) on both splits.

Split	Features	MAE ($)
Random 80/20	with quote_signal	38.5
Random 80/20	without quote_signal	51.6
Time: Jan-Aug -> Sep-Oct	with quote_signal	91.8
Time: Jan-Aug -> Sep-Oct	without quote_signal	117.6

The random split reports errors 2-3x lower than the time split; it would have selected the wrong features and overstated accuracy.

3.2 Rolling time-based folds

Fold	Train	Test	Train loads	Test loads
Jan-Apr -> May-Jun	01 Jan - 30 Apr	01 May - 30 Jun	19,110	9,696
Jan-Jun -> Jul-Aug	01 Jan - 30 Jun	01 Jul - 31 Aug	28,806	9,671
Jan-Aug -> Sep-Oct	01 Jan - 31 Aug	01 Sep - 31 Oct	38,477	9,523

- Each fold trains only on data before the test period and predicts the following two months, so the horizon matches Nov-Dec. Three folds give three independent tests across different market conditions.

- Models are compared on the mean and the worst fold, so a model that is lucky on one period is not preferred.
- Broken prices are removed from training only. Test scores are reported on clean loads (the model's real accuracy) and on all loads ('raw'), because the hidden validation labels may contain the same kind of errors.
- MAE is the main metric (dollars per load); RMSE and MAPE are reported too, as the scoring metric is not specified.
- After selection, the final model is retrained once on all clean Jan-Oct loads.

3.3 Leakage checks

- Lookup tables (daily market_index medians, city coordinates, weight medians) are built from input columns only, never from posted_rate. Validation inputs are used only to extend the daily market_index table to Nov-Dec, which needs no labels.
- The models and the market price level are fitted inside each fold from that fold's training data only.
- Broken prices are flagged once on all Jan-Oct loads. Flagging each fold's training data on its own picks exactly the same loads (275, 400 and 533), because no load sits between normal noise and the 1.8x cut-off.

3.4 The quote_signal trap

quote_signal changes behaviour week by week: in Jan-Mar, Jun and Sep it equals price per mile (correlation +0.996, a leak of the answer); in Apr, May, Jul and Oct it mirrors it (-0.99); in August it is noise (-0.02). Validation has no labels, but in every labelled month quote_signal is also strongly correlated with distance (because price per mile is), so that correlation shows the state without the answer. For November and December it is +0.02 and +0.01, the same as August (+0.01): noise. On August, a model using quote_signal scored $80 MAE against $36 without it, so the feature is dropped.

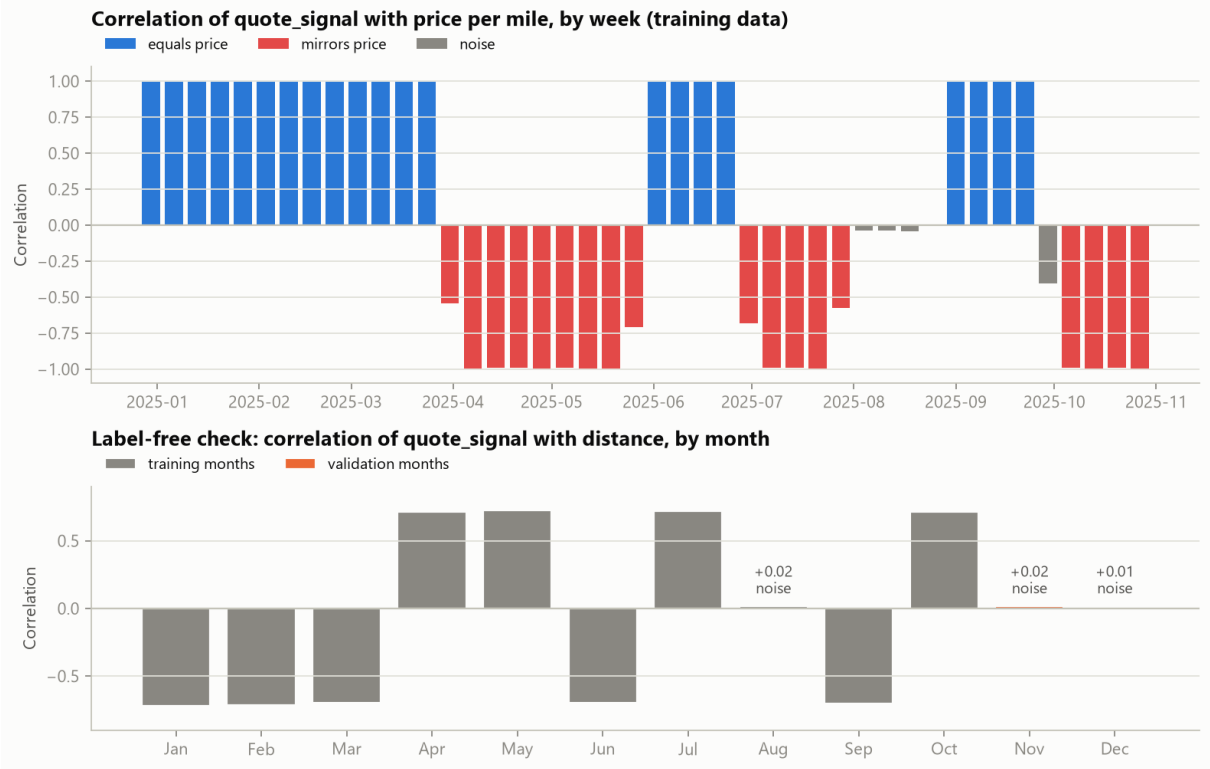


Figure 2. quote_signal by week (top) and the label-free check by month (bottom).

3.5 Robustness to unseen cities

Eight training cities (Bakersfield, Hartford, Memphis, New York, San Antonio, Toledo, Tulsa, Washington) were hidden from training and the models scored on loads touching them, mirroring the eight new cities in validation.

Model	MAE on unseen-city loads ($)
Log-linear (city one-hot)	70.1
LightGBM (coordinates)	56.9
Blend	57.0

4. Model

4.1 Load model

Two models predict log(rate) from the load's own attributes, and their predictions are averaged (70/30):

- LightGBM with these inputs: distance, log_distance, equipment, weight, weight_missing, market_index, pickup_lat, pickup_lon, delivery_lat, delivery_lon, straight_line_miles, delta_lat, delta_lon, day_of_week. City names are left out so it generalises to new cities; coordinates, straight-line distance and the lat/lon deltas carry the lane geography. Default-style settings (1,500 trees, learning rate 0.03, 31 leaves); tuning them changed backtest MAE by under $1.

- A log-linear Ridge model on log distance, log weight, log market_index, equipment, day of week and one-hot pickup and delivery cities. It captures the multiplicative structure cleanly; for an unseen city it falls back to no city effect.

4.2 Market price level

Prices for all loads on a given day move together in a way market_index does not explain. The model separates this daily level from the load model by alternating between the two fits, measures it on out-of-fold predictions (so the load model cannot have memorised the day), then describes it with three parts:

- Recent level: the average of the last 14 days of training, carried forward (+2.8% against the Jan-Oct average).
- Trend: prices drifted upward through the year by +0.59% a month; the trend is continued from the recent level.
- Quarter-end ramp: prices climb +4.1% over the last four weeks of each quarter and reset when the next starts (seen at the end of March, June and September). December ends a quarter.

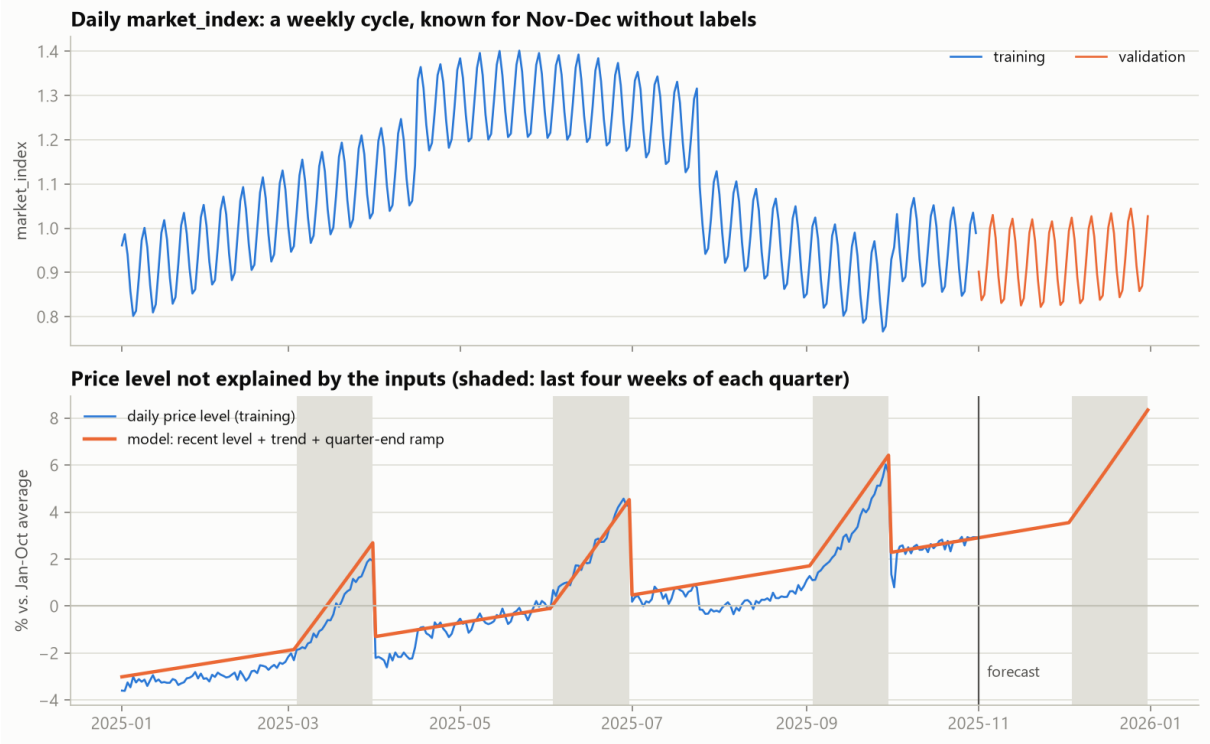


Figure 3. Daily market_index (top) and the estimated daily price level (bottom).

4.3 Model comparison

Model	MAE	Worst fold MAE	RMSE	MAPE	Bias	Raw MAE	Raw RMSE
Blend + market level	$44	$51	$65	1.89%	+1.2%	$98	$628
LightGBM + market level	$46	$55	$66	1.96%	+1.2%	$100	$629
LightGBM + recent level	$56	$62	$83	2.38%	+0.9%	$110	$631
Log-linear	$75	$115	$104	3.13%	+2.6%	$128	$637

LightGBM	$77	$118	$106	3.24%	+2.9%	$131	$638
Naive $/mile	$106	$148	$143	4.59%	+2.5%	$159	$645

Blend + market level is selected: it has the lowest mean and worst-fold MAE. It averages a log-linear model (clean multiplicative structure and city effects) with LightGBM (non-linear effects; coordinates for unseen cities), and both share one market price level of recent level + trend + quarter-end ramp.

Bias is the average log error on clean loads (positive means predictions too low). Raw RMSE is dominated by the broken prices, which no model can predict; on clean loads the error is about 2% of the rate.

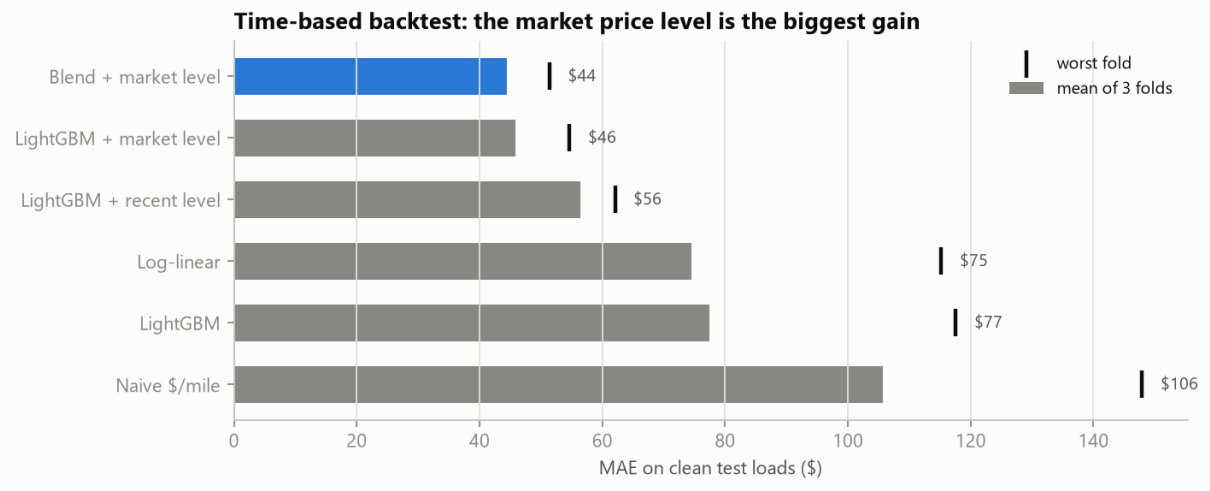


Figure 4. Mean MAE over the three folds (bar) and worst fold (tick).

5. December prediction chart

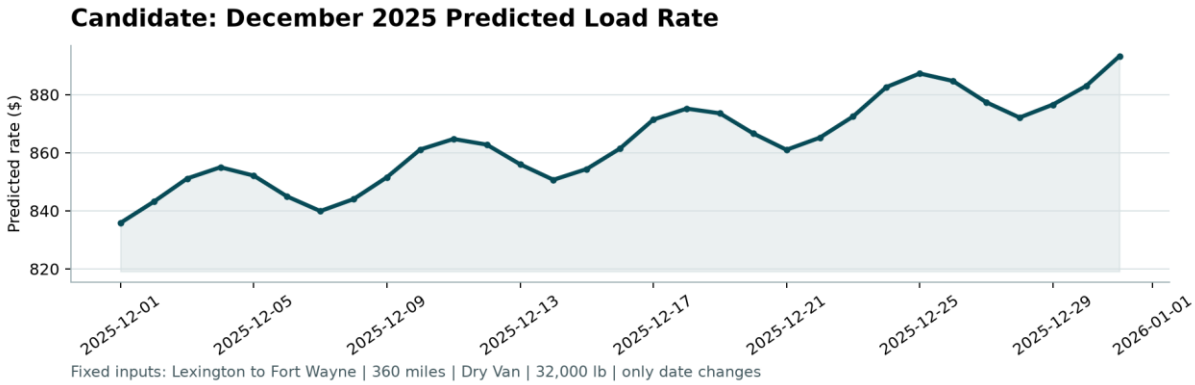


Figure 5. Chart produced by the provided score.py for the fixed December inputs.

Only the date changes (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb). Predictions run from $836 to $893. Each day uses that day's market_index from the validation inputs, which has a weekly cycle (peaking mid-week), so the line rises and falls weekly. From early December the quarter-end

ramp lifts prices towards 31 December. The 32 past loads on this lane (all equipment types) were priced between about $790 and $920 per month on median. No holiday-specific effect is assumed: training contains no December and showed no effect around Memorial Day or 4 July.

6. Limitations and next steps

- The scoring metric is not stated. The model targets typical (median-like) error in log space; if raw RMSE is used, broken prices in the validation labels will dominate any model's score.
- Nov-Dec level is a forecast. The trend and quarter-end ramp are extrapolated from three quarters; a year-end holiday effect, if any, cannot be learned from this data. In production the level would be re-estimated weekly from the latest posted rates.
- The selected model under-predicted by +0.7% to +1.8% across the three folds: prices rose a little faster than the straight-line trend. No correction factor was added, because three folds are too few to estimate one reliably; a damped or piecewise trend is the next thing to test.
- Next steps: prediction intervals (quantile LightGBM) for pricing risk; lane-level history features once more data exists; find the source of quote_signal and fix the pipeline that produces it.

Appendix: reproduce

```
python -m pip install -r requirements-dev.txt

python -m src.train

python -m src.diagnostics

python -m src.predict

python score.py --predictions validation_predictions.csv --december-predictions
data/december_chart_inputs.csv

python -m pytest
```